

線上訂餐系統

資料庫系統期末報告

組別：G06

組員：41243118 李銘杰 41243121 林泓宇

41243146 黃境安 41243122 林建全

日期：2026 年 6 月 21 日

目錄

目錄.....	II
圖目錄.....	IV
表目錄.....	V
摘要.....	VI
一、應用情境.....	1
顧客.....	1
餐廳.....	1
二、系統需求說明.....	2
三、使用案例.....	3
四、資料庫 ER 設計	4
五、完整性限制.....	6
(一) 實體完整性限制	6
(二) 參照完整性限制	7
(三) 欄位完整性限制	8
六、View 與查詢設計	9
七、使用者權限.....	15
八、操作範例.....	18
新增訂單.....	18
新增與修改餐點.....	18
刪除訂單明細與餐點.....	18
九、分工.....	19
十、結論與心得.....	20
參考資料.....	22
附錄.....	23
Customer	23

Restaurant	23
Meal	24
OrderDetail	24
Orders.....	25
Payment.....	25

圖目錄

圖 1使用 案例圖.....	3
圖 2 ER Diagram 詳圖	4
圖 3顧客菜單 View 查詢結果.....	9
圖 4顧客菜單 View 查詢結果.....	9
圖 5顧客訂單紀錄 View 查詢結果.....	10
圖 6顧客訂單紀錄 View 查詢結果.....	10
圖 7店家訂單管理 View 查詢結果.....	11
圖 8店家訂單管理 View 查詢結果.....	11
圖 9店家修改商品狀態.....	12
圖 10店家修改商品狀態後資料表更新完的狀態.....	12
圖 11店家修改商品狀態後資料表更新完的狀態.....	12
圖 12店家修改價格.....	12
圖 13店家修改價格後資料表更新完的狀態.....	13
圖 14店家修改價格後資料表更新完的狀態.....	13
圖 15刪除定單明細.....	13
圖 16刪除定單明細後資料表更新完的狀態.....	14
圖 17刪除定單明細後資料表更新完的狀態.....	14
圖 18顧客角色權限設定.....	16
圖 19顧客角色權限設定.....	16
圖 20店家角色權限設定.....	17
圖 21店家角色權限設定.....	17

表目錄

表 1 ERD 關係表.....	5
表 2實體完整性限制表.....	6
表 3參照完整性限制表.....	7
表 4欄位完整性限制表.....	8
表 5使用者權限.....	15

摘要

本報告以線上訂餐系統為主題，規劃顧客、餐廳、餐點、訂單、訂單明細與付款等資料表，並以主鍵、外鍵、CHECK 條件與 ENUM 狀態欄位維持資料完整性。系統支援顧客瀏覽餐點、建立訂單、付款與查詢訂單，也提供店家餐點管理、訂單狀態維護與帳號資料管理。整體資料庫設計透過 ER Diagram 表達實體關係，並設計 View 與使用者權限，讓不同角色只能操作符合需求的資料範圍。

一、應用情境

顧客

- 顧客進入線上訂餐系統後，可以瀏覽目前營業中的餐廳，查看餐點名稱、價格、說明與販售狀態。顧客選擇餐點與數量後，系統會建立訂單與訂單明細，記錄顧客、餐廳、餐點、數量、金額與訂單狀態。
- 顧客確認訂單後，可選擇現金、信用卡或 LINE Pay 付款。系統會建立付款資料，記錄付款方式、付款狀態、付款金額與交易資訊；付款完成後，訂單狀態同步更新，方便後續查詢與餐廳處理。

餐廳

- 餐廳進入系統後，可以管理餐廳基本資料與營業狀態。當餐廳狀態為 Open 時，顧客可以瀏覽並訂餐；若狀態為 Closed，則不接受訂單。
- 餐廳也可以管理餐點資料，包括新增、修改餐點名稱、價格、介紹與販售狀態。顧客送出訂單後，餐廳可查看訂單內容、付款狀態與訂單進度，並依處理狀況更新為已完成或已取消。

二、系統需求說明

顧客：

- 瀏覽與搜尋餐點
 - 顧客可以瀏覽目前可販售的餐點，並依餐廳或餐點名稱搜尋想要的餐點。
- 查看餐點詳細資訊
 - 顧客可以查看餐點名稱、價格、餐點介紹、所屬餐廳與販售狀態。
- 新增訂單
 - 顧客選擇餐點與數量後，系統會建立訂單與訂單明細，並記錄訂單狀態與建立時間。
- 查詢訂單紀錄
 - 顧客可以查詢自己過去建立的訂單，查看訂單內容、付款狀態與目前處理進度。
- 選擇付款方式
 - 顧客確認訂單後，可以選擇現金、信用卡或 LINE Pay 付款，系統會建立付款紀錄並更新付款狀態。
- 取消未付款訂單
 - 顧客可以取消尚未付款的訂單，系統會將訂單狀態更新為已取消；若訂單已付款或已完成，則不可取消。

店家：

- 餐點管理
 - 店家可新增、修改、停用餐點，並設定價格、介紹與販售狀態。
- 訂單管理
 - 店家可查看訂單內容、付款狀態，並更新訂單處理狀態。
- 店家資訊管理
 - 店家可管理帳號資料、店名、電話、地址與營業狀態。

三、使用案例

依照系統需求，本系統主要參與者可分為顧客與店家。顧客重點在訂餐與付款；店家重點在餐點、訂單與餐廳狀態管理。

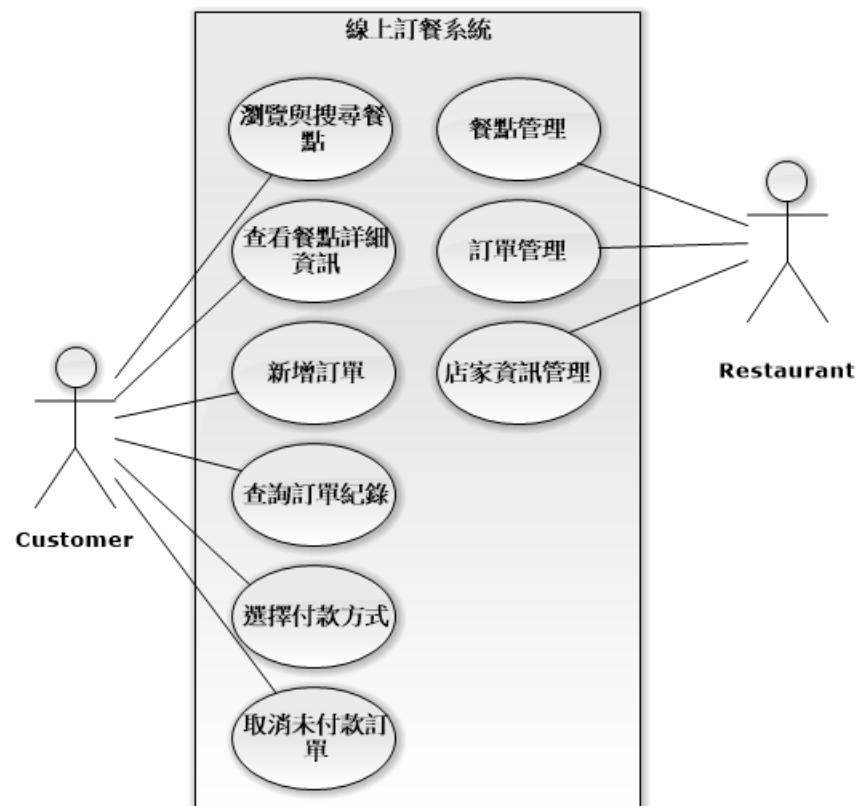


圖 1 使用案例圖

四、資料庫 ER 設計

資料庫核心實體包含 Customer、Restaurant、Meal、Orders、OrderDetail 與 Payment。顧客可建立多筆訂單，餐廳可提供多項餐點並接收多筆訂單；訂單透過 OrderDetail 記錄多個餐點項目，並由 Payment 記錄付款資訊。

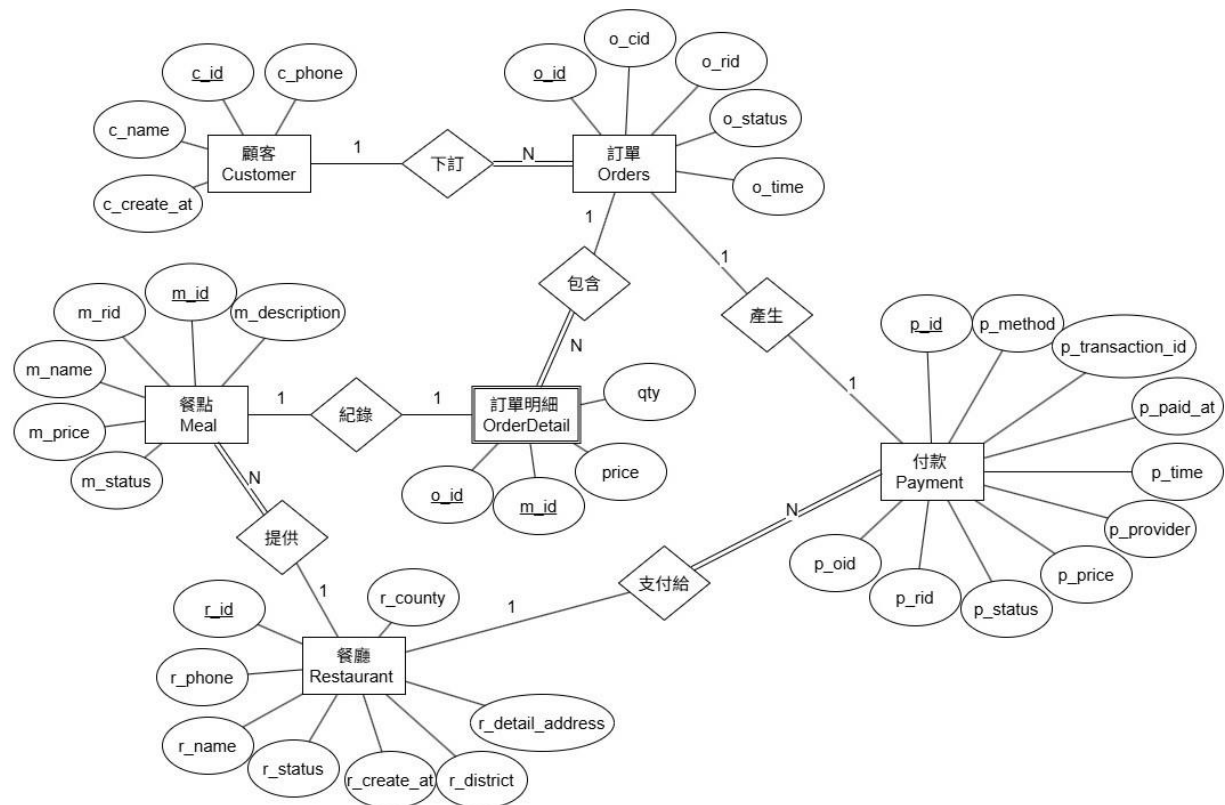


圖 2 ER Diagram 詳圖

表 1 ERD 關係表

關係	基數	設計意義
Customer - Orders	1:N	一位顧客可建立多筆訂單。
Restaurant - Meal	1:N	一家餐廳可提供多項餐點。
Payment - Restaurant	N:1	一家餐廳可接收多筆付款。
Orders - OrderDetail	1:N	一筆訂單可包含多個餐點明細。
Meal - OrderDetail	1:1	同一餐點對應到一筆訂單明細中。
Orders - Payment	1:1	一筆訂單對應一筆付款紀錄。

五、完整性限制

(一) 實體完整性限制

表 2 實體完整性限制表

資料表	主鍵欄位	說明
Customer	c_id	顧客識別碼，格式為 C 加 0~9 的 6 位數字
Orders	o_id	訂單識別碼，15 位數字
OrderDetail	o_id, m_id	複合主鍵，避免同一張訂單重複記錄同一餐點
Meal	m_id	餐點識別碼，格式為 M 加 2 位數字
Payment	p_id	付款編號，格式為 P 加 15 位數字
Restaurant	r_id	餐廳識別碼，3 位數字

(二) 參照完整性限制

表 3參照完整性限制表

子資料表	外鍵欄位	參照資料表	參照欄位	說明
Orders	o_cid	Customer	c_id	一筆訂單必須屬於已存在的顧客
Orders	o_rid	Restaurant	r_id	一筆訂單必須屬於已存在的餐廳
OrderDetail	o_id	Orders	o_id	訂單明細必須對應已存在的訂單
OrderDetail	m_id	Meal	m_id	訂單明細中的餐點必須存在
Meal	m_rid	Restaurant	r_id	餐點必須屬於已存在的餐廳
Payment	p_rid	Restaurant	r_id	付款資料需對應收款餐廳
Payment	p_oid	Orders	o_id	付款資料需對應已存在的訂單

(三) 欄位完整性限制

表 4 欄位完整性限制表

資料表	欄位	值域限制
Customer	c_id	CHAR(7)，格式 ^C[0-9]{6}\$
Customer	c_name	CHAR(50)，不可為空，只能為英文或中文
Customer	c_phone	CHAR(10)，格式 ^09[0-9]{8}\$
Orders	o_status	ENUM('PENDING','PAID','CANCELLED','COMPLETED') ，預設 PENDING
OrderDetail	price	INT，不可小於 0
OrderDetail	qty	INT，必須大於 0
Meal	m_id	CHAR(3)，格式 ^M[0-9]{2}\$
Meal	m_price	INT，必須大於 0
Meal	m_status	ENUM('Enable','Disable')，預設 Disable
Payment	p_method	ENUM('Cash','CreditCard','LINE Pay')
Payment	p_status	ENUM('Pending','Paid','Failed')，預設 Pending
Payment	p_price	INT，付款金額必須大於 0
Payment	p_transaction_id	VARCHAR(100)，第三方交易編號需唯一
Restaurant	r_id	CHAR(3)，格式 ^[0-9]{3}\$
Restaurant	r_phone	CHAR(10)，格式 ^0[0-9]{8,9}\$
Restaurant	r_status	ENUM('Open','Closed')，預設 Closed
Restaurant	county, district, detail_address	不可為空字串

六、View 與查詢設計

顧客瀏覽餐點，顯示餐點名稱、價格、介紹等狀態

```
MariaDB [food_db]> SELECT * FROM View_Customer_Menu;
```

餐點編號	餐點名稱	價格	餐點介紹
M01	招牌起司牛肉堡	120	經典熱銷款
M02	酥脆炸薯條	50	現炸好吃
M03	夏威夷披薩	250	酸甜好滋味
M04	海鮮總匯披薩	300	豐富海鮮
M06	沙朗厚切牛排	220	夜市神級美食
M07	火腿蛋吐司	40	NULL
M09	招牌滷肉飯	60	肥而不膩

圖 3顧客菜單 View 查詢結果

販售狀態	餐廳編號	所屬餐廳	餐廳營業狀態
Enable	001	斗六美味漢堡	Open
Enable	001	斗六美味漢堡	Open
Enable	002	雲林快樂披薩	Open
Enable	002	雲林快樂披薩	Open
Enable	004	虎尾夜市牛排	Open
Enable	005	早安美芝城	Open
Enable	007	阿明魯肉飯	Open

圖 4顧客菜單 View 查詢結果

顧客查看歷史訂單紀錄

```
MariaDB [food_db]> SELECT*FROM View_Customer_Order_History
-> ;
```

顧客編號	訂單編號	建立時間	餐廳名稱	餐點名稱
C000001	001202606190001	2026-06-19 12:00:00	斗六美味漢堡	招牌起司牛肉堡
C000002	001202606190002	2026-06-19 12:15:00	斗六美味漢堡	酥脆炸薯條
C000003	002202606190003	2026-06-19 12:30:00	雲林快樂披薩	夏威夷披薩
C000004	002202606190004	2026-06-19 12:45:00	雲林快樂披薩	海鮮總匯披薩
C000005	003202606190005	2026-06-19 13:00:00	斗南拉麵道	豚骨拉麵
C000006	004202606190006	2026-06-19 18:00:00	虎尾夜市牛排	沙朗厚切牛排
C000007	005202606190007	2026-06-19 08:00:00	早安美芝城	火腿蛋吐司
C000008	006202606190008	2026-06-19 19:00:00	鼎泰豐	小籠包
C000009	007202606190009	2026-06-19 12:30:00	阿明魯肉飯	招牌滷肉飯
C000010	008202606190010	2026-06-19 15:00:00	星巴克	焦糖瑪奇朵

圖 5顧客訂單紀錄 View 查詢結果

數量	明細小計	處理進度	付款方式	付款狀態
2	240	COMPLETED	Cash	Paid
1	50	COMPLETED	LINE Pay	Paid
1	250	PAID	CreditCard	Paid
2	600	PENDING	Cash	Pending
1	180	CANCELLED	LINE Pay	Failed
3	660	COMPLETED	Cash	Paid
1	40	COMPLETED	Cash	Paid
2	500	PAID	CreditCard	Paid
4	240	COMPLETED	LINE Pay	Paid
1	150	PENDING	CreditCard	Pending

圖 6顧客訂單紀錄 View 查詢結果

```
MariaDB [food_db]> SELECT*FROM View_Store_Order_Management
-> ;
```

餐廳編號	訂單編號	下單時間	顧客姓名	聯絡電話
001	001202606190001	2026-06-19 12:00:00	王小明	0912345671
001	001202606190002	2026-06-19 12:15:00	林美玲	0922345672
002	002202606190003	2026-06-19 12:30:00	陳大毛	0932345673
002	002202606190004	2026-06-19 12:45:00	張阿呆	0942345674
003	003202606190005	2026-06-19 13:00:00	李心潔	0952345675
004	004202606190006	2026-06-19 18:00:00	趙麗穎	0962345676
005	005202606190007	2026-06-19 08:00:00	孫燕姿	0972345677
006	006202606190008	2026-06-19 19:00:00	周杰倫	0982345678
007	007202606190009	2026-06-19 12:30:00	蔡依林	0992345679
008	008202606190010	2026-06-19 15:00:00	林俊傑	0902345670

圖 7店家訂單管理 View 查詢結果

餐點名稱	數量	訂單狀態	付款方式	付款狀態
招牌起司牛肉堡	2	COMPLETED	Cash	Paid
酥脆炸薯條	1	COMPLETED	LINE Pay	Paid
夏威夷披薩	1	PAID	CreditCard	Paid
海鮮總匯披薩	2	PENDING	Cash	Pending
豚骨拉麵	1	CANCELLED	LINE Pay	Failed
沙朗厚切牛排	3	COMPLETED	Cash	Paid
火腿蛋吐司	1	COMPLETED	Cash	Paid
小籠包	2	PAID	CreditCard	Paid
招牌滷肉飯	4	COMPLETED	LINE Pay	Paid
焦糖瑪奇朵	1	PENDING	CreditCard	Pending

圖 8店家訂單管理 View 查詢結果

```

MariaDB [food_db]> UPDATE Meal
    -> SET m_status = 'Disable'
    -> WHERE m_id = 'M02' AND m_rid = '001'
Query OK, 1 row affected (0.002 sec)
Rows matched: 1  Changed: 1  Warnings: 0

```

圖 9 店家修改商品狀態

```

MariaDB [food_db]> SELECT * FROM View_Customer_Menu
    -> WHERE 餐廳編號 = '001';

```

餐點編號	餐點名稱	價格	餐點介紹
M01	招牌起司牛肉堡	130	經典熱銷款
M11	超級雙層牛肉堡	180	NULL

圖 10 店家修改商品狀態後資料表更新完的狀態

販售狀態	餐廳編號	所屬餐廳	餐廳營業狀態
Enable	001	斗六美味漢堡	Open
Enable	001	斗六美味漢堡	Open

圖 11 店家修改商品狀態後資料表更新完的狀態

```

MariaDB [food_db]> UPDATE Meal
    -> SET m_price = 130
    -> WHERE m_id = 'M01' AND m_rid = '001';
Query OK, 1 row affected (0.005 sec)
Rows matched: 1  Changed: 1  Warnings: 0

```

圖 12 店家修改價格

```
MariaDB [food_db]> SELECT * FROM View_Customer_Menu
-> WHERE 餐廳編號 = '001';
```

餐點編號	餐點名稱	價格	餐點介紹
M01	招牌起司牛肉堡	130	經典熱銷款
M02	酥脆炸薯條	50	現炸好吃
M11	超級雙層牛肉堡	180	NULL

圖 13店家修改價格後資料表更新完的狀態

販售狀態	餐廳編號	所屬餐廳	餐廳營業狀態
Enable	001	斗六美味漢堡	Open
Enable	001	斗六美味漢堡	Open
Enable	001	斗六美味漢堡	Open

圖 14店家修改價格後資料表更新完的狀態

```
MariaDB [food_db]> -- 1. 先刪除所有與該商品相關的訂單明細
MariaDB [food_db]> DELETE FROM OrderDetail
-> WHERE m_id = 'M02';
Query OK, 2 rows affected (0.003 sec)

MariaDB [food_db]>
MariaDB [food_db]> -- 2. 這時候商品沒有被任何人參考了，可以安全地直接刪除
MariaDB [food_db]> DELETE FROM Meal
-> WHERE m_id = 'M02' AND m_rid = '001';
Query OK, 1 row affected (0.001 sec)

MariaDB [food_db]> SELECT * FROM OrderDetail
-> WHERE m_id = 'M02';
Empty set (0.001 sec)
```

圖 15刪除定單明細

```
MariaDB [food_db]> SELECT * FROM Meal;
```

m_id	m_rid	m_name	m_price	m_description	m_status
M01	001	招牌起司牛肉堡	130	經典熱銷款	Enable
M03	002	夏威夷披薩	250	酸甜好滋味	Enable
M04	002	海鮮總匯披薩	300	豐富海鮮	Enable
M05	003	豚骨拉麵	180	濃郁大骨湯頭	Enable
M06	004	沙朗厚切牛排	220	夜市神級美食	Enable
M07	005	火腿蛋吐司	40	NULL	Enable
M08	006	小籠包	250	米其林推薦	Enable
M09	007	招牌滷肉飯	60	肥而不膩	Enable
M10	008	焦糖瑪奇朵	150	特大杯	Disable
M11	001	超級雙層牛肉堡	180	NULL	Enable

圖 16刪除定單明細後資料表更新完的狀態

o_id	m_id	price	qty
001202606190001	M01	120	2
001202606210001	M01	120	2
002202606190003	M03	250	1
002202606190004	M04	300	2
003202606190005	M05	180	1
004202606190006	M06	220	3
005202606190007	M07	40	1
006202606190008	M08	250	2
007202606190009	M09	60	4
008202606190010	M10	150	1

圖 17刪除定單明細後資料表更新完的狀態

七、使用者權限

本系統以角色區分顧客與店家的資料庫權限。顧客角色主要能查詢餐廳與餐點、建立訂單與付款；店家角色主要能維護餐廳、餐點與訂單狀態。此設計可降低使用者直接操作非授權資料表的風險。

表 5 使用者權限

資料表 (Table)	顧客角色 (role_customer)	店家角色 (role_restaurant)	應用層級必須加上的限制 (WHERE 條件)
Customer (顧客)	SELECT, UPDATE	SELECT	顧客：c_id = 自己的 ID 店家：只能透過訂單 JOIN 查詢，不可盲搜
Restaurant (餐廳)	SELECT	SELECT, UPDATE	顧客：無限制（可看全部） 店家：r_id = 自己的店家 ID
Meal (餐點)	SELECT	SELECT, INSERT, UPDATE, DELETE	顧客：m_status = 'Enable' 店家：m_rid = 自己的店家 ID
Orders (訂單)	SELECT, INSERT, UPDATE	SELECT, UPDATE	顧客：o_cid = 自己的 ID 店家：o_rid = 自己的店家 ID
OrderDetail (明細)	SELECT, INSERT	SELECT	顧客：綁定自己的 o_id 店家：綁定本店的 o_id
Payment (付款)	SELECT, INSERT	SELECT, UPDATE	顧客：綁定自己的 o_id 店家：p_rid = 自己的店家 ID

```
MariaDB [food_db]> GRANT SELECT, UPDATE ON food_db.Customer TO 'role_customer';
Query OK, 0 rows affected (0.003 sec)

MariaDB [food_db]> GRANT SELECT ON food_db.Restaurant TO 'role_customer';
Query OK, 0 rows affected (0.001 sec)

MariaDB [food_db]> GRANT SELECT ON food_db.Meal TO 'role_customer';
Query OK, 0 rows affected (0.001 sec)

MariaDB [food_db]> GRANT SELECT, INSERT, UPDATE ON food_db.Orders TO 'role_customer';
Query OK, 0 rows affected (0.001 sec)

MariaDB [food_db]> GRANT SELECT, INSERT ON food_db.OrderDetail TO 'role_customer';
Query OK, 0 rows affected (0.001 sec)

MariaDB [food_db]> GRANT SELECT, INSERT ON food_db.Payment TO 'role_customer';
Query OK, 0 rows affected (0.001 sec)
```

圖 18顧客角色權限設定

```
MariaDB [food_db]> CREATE USER IF NOT EXISTS 'app_customer_user'@'localhost' IDENTIFIED BY 'CustomerPwd123!';
Query OK, 0 rows affected (0.007 sec)

MariaDB [food_db]> GRANT 'role_customer' TO 'app_customer_user'@'localhost';
Query OK, 0 rows affected (0.001 sec)

MariaDB [food_db]> SET DEFAULT ROLE 'role_customer' FOR 'app_customer_user'@'localhost';
Query OK, 0 rows affected (0.003 sec)
```

圖 19顧客角色權限設定

```
MariaDB [food_db]> GRANT SELECT ON food_db.Customer TO 'role_restaurant';
Query OK, 0 rows affected (0.002 sec)

MariaDB [food_db]> GRANT SELECT, UPDATE ON food_db.Restaurant TO 'role_restaurant';
Query OK, 0 rows affected (0.001 sec)

MariaDB [food_db]> GRANT SELECT, INSERT, UPDATE, DELETE ON food_db.Meal TO 'role_restaurant';
Query OK, 0 rows affected (0.001 sec)

MariaDB [food_db]> GRANT SELECT, UPDATE ON food_db.Orders TO 'role_restaurant';
Query OK, 0 rows affected (0.000 sec)

MariaDB [food_db]> GRANT SELECT ON food_db.OrderDetail TO 'role_restaurant';
Query OK, 0 rows affected (0.000 sec)

MariaDB [food_db]> GRANT SELECT, UPDATE ON food_db.Payment TO 'role_restaurant';
Query OK, 0 rows affected (0.002 sec)
```

圖 20 店家角色權限設定

```
MariaDB [food_db]> CREATE USER IF NOT EXISTS 'app_restaurant_user'@'localhost' IDENTIFIED BY 'StorePwd123!';
Query OK, 0 rows affected (0.003 sec)

MariaDB [food_db]> GRANT 'role_restaurant' TO 'app_restaurant_user'@'localhost';
Query OK, 0 rows affected (0.001 sec)

MariaDB [food_db]> SET DEFAULT ROLE 'role_restaurant' FOR 'app_restaurant_user'@'localhost';
Query OK, 0 rows affected (0.001 sec)
```

圖 21 店家角色權限設定

八、操作範例

簡報中的實作範例包含新增訂單、餐點新增與修改，以及訂單明細與餐點刪除。這些操作可驗證資料表關聯與權限設定是否正常運作。

新增訂單

```
INSERT INTO Orders (o_id, o_cid, o_rid, o_status)
VALUES ('001202606210001', 'C000001', '001', 'PENDING');

INSERT INTO OrderDetail (o_id, m_id, price, qty)
VALUES
('001202606210001', 'M01', 120, 2),
('001202606210001', 'M02', 50, 1);
```

新增與修改餐點

```
INSERT INTO Meal (m_id, m_rid, m_name, m_price, m_status)
VALUES ('M11', '001', '超級雙層牛肉堡', 180, 'Enable');

UPDATE Meal
SET m_price = 130
WHERE m_id = 'M01' AND m_rid = '001';
```

刪除訂單明細與餐點

```
DELETE FROM OrderDetail
WHERE m_id = 'M02';

DELETE FROM Meal
WHERE m_id = 'M02' AND m_rid = '001';
```

九、分工

成員	學號	專題負責部分
李銘杰	41243118	
林泓宇	41243121	ER 圖、資料庫架構設計
林建全	41243122	資料庫建置、VIEW
黃境安	41243146	ER 圖、資料庫架構設計

十、結論與心得

林建全：

在這次線上訂餐系統的資料庫專題中，我深刻體會到系統設計的核心不僅在於條列功能，更在於精準捕捉使用者的真實需求與商業邏輯。初期在定義功能時，我們曾誤將「登入」等系統基本流程視為主功能，經釐清後才明白，真正的系統價值應聚焦於點餐、接單等核心業務的流轉。在架構設計階段，我們於繪製 ERD（實體關聯圖）時，面臨了實體定義與關聯性模糊的挑戰。經過不斷地討論與反覆修正，才成功梳理出「顧客、店家、餐點、訂單與付款」等核心實體間的緊密連結，讓整體的資料流邏輯更趨合理。此外，本次專題最大的技術收穫在於對「資料限制條件（Constraints）」的實務應用。過去我對欄位的認知僅停留在資料型態，但透過這次實作，我真正理解了主鍵（PK）、外鍵（FK）、非空值（NOT NULL）、條件檢查（CHECK）以及列舉（ENUM）等約束條件的重要性。這些機制不僅是語法，更是確保資料完整性、防範「幽靈資料」產生的關鍵防線。

黃境安：

這次資料庫專題讓我發現，設計一個系統不只是把功能列出來而已，還必須思考功能是否真的符合使用者的需求。一開始我們在功能定義時把「登入」寫成主要功能，但後來才知道登入比較偏向系統基本流程，不應該取代真正的業務功能。除此之外，在設計 ERD 時也遇到很多問題，例如實體之間的關係不夠清楚、實體定義不完整，導致整體邏輯不夠合理。後來經過修正後，才逐漸理解顧客、店家、餐點、訂單與付款之間應該如何連結。另外，屬性的限制條件也是這次學到最多的部分。原本只知道欄位要有資料型態，但沒有特別注意主鍵、外鍵、NOT NULL、CHECK、ENUM 等限制的重要性。經過這次專題，我更了解資料庫設計需要兼顧完整性與實際使用情境，也學到在設計前應該先把需求分析清楚，才能建立起比較合理且完整的資料庫架構。

林泓宇:

在這堂課中，我從資料庫的基礎概念開始學習，了解資料庫並不是單純把資料存進表格中，而是需要依照實際需求進行規劃與設計。從資料表的欄位安排、資料型態設定，到實際使用 SQL 語法建立資料庫，這些過程都讓我學到許多資料處理上的細節。最初我沒有想到設計資料庫需要注意每一項欄位的限制，原本認為只要依照欄位需求給出相對應的值就可以了，但在實務上，還需要考慮各種可能發生的狀況，才能避免資料錯誤或系統運作上的問題。有了這些基礎後，我們進行資料庫專題，並選擇從頭設計線上點餐系統。我們參考了常見點餐平台具備的功能，將顧客、餐點、訂單、付款、購物車等功能所需要的欄位規劃成資料表，並進一步完善資料表之間的結構，以及主鍵與外鍵之間的關係。在這次專題中，我印象最深的是完整性限制的部分。設計資料庫時，必須清楚限制欄位能夠輸入的文字或數值，例如訂單狀態、付款方式、餐點價格與數量等欄位，都需要設定合理的限制，才能避免資料輸入錯誤。這些限制雖然在設計時需要花費較多時間思考，但都是為了讓資料庫能夠正常運作，並降低系統發生錯誤的情況。

參考資料

- [1] 虎尾野餐日線上訂餐網站. (n.d.). iCHEF POS. Retrieved June 21, 2026, from <https://shop.ichefpos.com/store/9iSDXw2e/ordering>
- [2] 八方雲集門市查詢. (n.d.). 八方雲集. Retrieved June 21, 2026, from <https://www.8way.com.tw/store>

附錄

Customer

```
CREATE TABLE Customer (  
  c_id CHAR(7) NOT NULL,  
  c_name CHAR(50) NOT NULL,  
  c_phone CHAR(10) NOT NULL,  
  c_create_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (c_id),  
  CHECK (c_id REGEXP '^C[0-9]{6}'),  
  CHECK (c_name REGEXP '^[A-Za-z ー-顚]+'),  
  CHECK (LENGTH(TRIM(c_name)) > 0),  
  CHECK (c_phone REGEXP '^09[0-9]{8}$')  
);
```

Restaurant

```
CREATE TABLE Restaurant (  
  r_id CHAR(3) NOT NULL,  
  r_name CHAR(20) NOT NULL,  
  r_phone CHAR(10) NOT NULL,  
  r_status ENUM('Open', 'Closed') NOT NULL DEFAULT 'Closed',  
  county CHAR(10) NOT NULL,  
  district CHAR(10) NOT NULL,  
  detail_address VARCHAR(50) NOT NULL,  
  r_create_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (r_id),  
  CHECK (r_id REGEXP '^[0-9]{3}'),  
  CHECK (r_name REGEXP '^[A-Za-z ー-顚]+'),  
  CHECK (LENGTH(TRIM(r_name)) > 0),  
  CHECK (r_phone REGEXP '^0[0-9]{8,9}$'),  
  CHECK (LENGTH(TRIM(county)) > 0),  
  CHECK (LENGTH(TRIM(district)) > 0),  
  CHECK (LENGTH(TRIM(detail_address)) > 0)  
);
```

Meal

```
CREATE TABLE Meal (  
  m_id CHAR(3) NOT NULL,  
  m_rid CHAR(3) NOT NULL,  
  m_name CHAR(20) NOT NULL,  
  m_price INT NOT NULL,  
  m_description VARCHAR(255) NULL,  
  m_status ENUM('Enable', 'Disable') NOT NULL DEFAULT 'Disable',  
  PRIMARY KEY (m_id),  
  FOREIGN KEY (m_rid) REFERENCES Restaurant(r_id),  
  CHECK (m_id REGEXP '^M[0-9]{2}'),  
  CHECK (m_name REGEXP '^[A-Za-z 一-龠]+'),  
  CHECK (LENGTH(TRIM(m_name)) > 0),  
  CHECK (m_price > 0)  
);
```

OrderDetail

```
CREATE TABLE OrderDetail (  
  o_id CHAR(15) NOT NULL,  
  m_id CHAR(3) NOT NULL,  
  price INT NOT NULL,  
  qty INT NOT NULL,  
  PRIMARY KEY (o_id, m_id),  
  FOREIGN KEY (o_id) REFERENCES Orders(o_id),  
  FOREIGN KEY (m_id) REFERENCES Meal(m_id),  
  CHECK (price >= 0),  
  CHECK (qty > 0)  
);
```

Orders

```
CREATE TABLE Orders (  
  o_id CHAR(15) NOT NULL,  
  o_cid CHAR(7) NOT NULL,  
  o_rid CHAR(3) NOT NULL,  
  o_status ENUM('PENDING', 'PAID', 'CANCELLED', 'COMPLETED') NOT NULL  
  DEFAULT 'PENDING',  
  o_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (o_id),  
  FOREIGN KEY (o_cid) REFERENCES Customer(c_id),  
  FOREIGN KEY (o_rid) REFERENCES Restaurant(r_id),  
  CHECK (o_id REGEXP '^[0-9]{15}$')  
);
```

Payment

```
CREATE TABLE Payment (  
  p_id CHAR(16) NOT NULL,  
  p_rid CHAR(3) NOT NULL,  
  p_oid CHAR(15) NOT NULL,  
  p_method ENUM('Cash', 'CreditCard', 'LINE Pay') NOT NULL,  
  p_status ENUM('Pending', 'Paid', 'Failed') NOT NULL DEFAULT 'Pending',  
  p_price INT NOT NULL,  
  p_transaction_id VARCHAR(100) UNIQUE,  
  p_provider CHAR(20) NULL,  
  p_paid_at DATETIME,  
  p_time DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  PRIMARY KEY (p_id),  
  FOREIGN KEY (p_rid) REFERENCES Restaurant(r_id),  
  FOREIGN KEY (p_oid) REFERENCES Orders(o_id),  
  CHECK (p_id REGEXP '^P[0-9]{15}$'),  
  CHECK (p_price > 0),  
  CHECK ((p_method = 'Cash' AND p_transaction_id IS NULL)  
    OR (p_method IN ('CreditCard', 'LINE Pay') AND p_transaction_id IS NOT NULL)),  
  CHECK ((p_status = 'Paid' AND p_paid_at IS NOT NULL)  
    OR (p_status <> 'Paid' AND p_paid_at IS NULL))  
);
```